

Prolog Program to Implement Towers of Hanoi

1. Aim

To write and execute a Prolog program to solve the Towers of Hanoi puzzle for N disks and display the sequence of moves required.

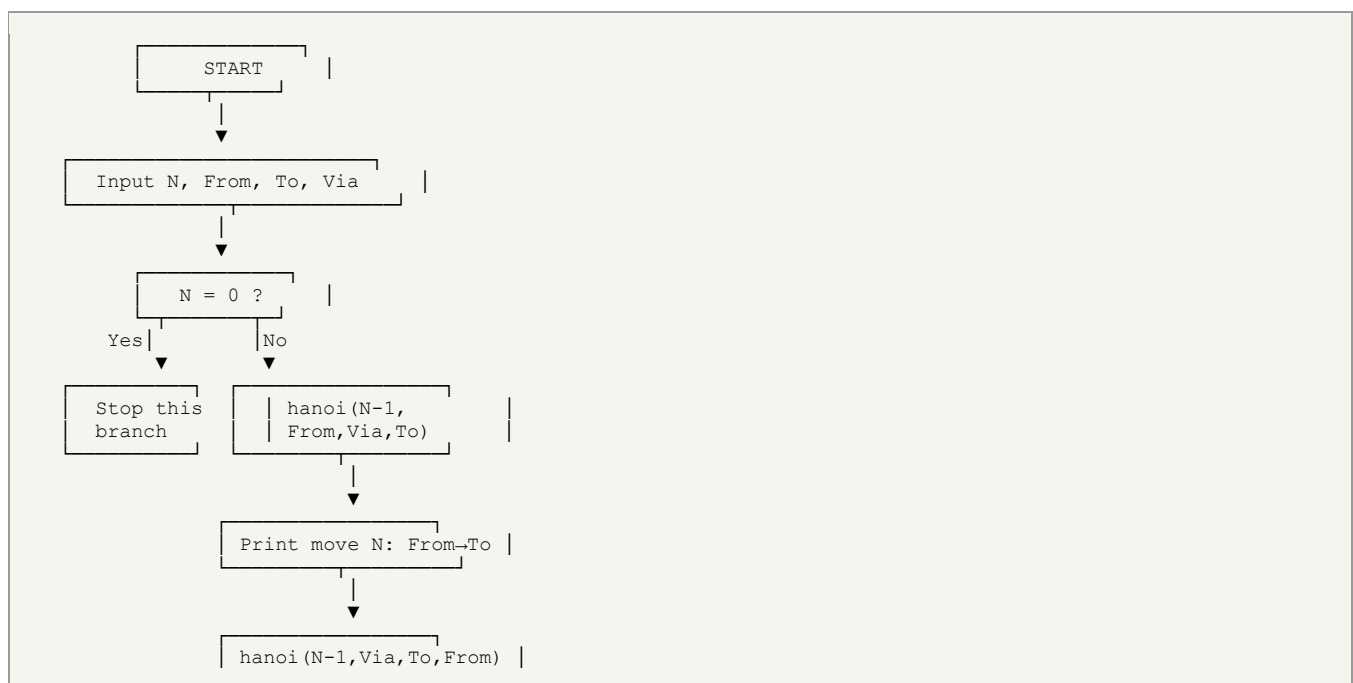
2. Objective

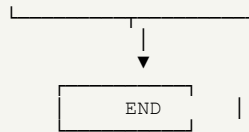
- To understand recursion in Prolog.
- To represent the Towers of Hanoi problem using a recursive predicate.
- To generate and display the sequence of disk moves between pegs.
- To understand the base case and recursive case in a Prolog recursive rule.

3. Algorithm

1. Start the program.
2. Define the predicate hanoi/4 taking the number of disks and the source, destination, and auxiliary pegs.
3. Base case: if the number of disks is 0, do nothing.
4. Recursive case: move N-1 disks from source to auxiliary peg (using destination as helper).
5. Print the move of disk N from source peg to destination peg.
6. Move the N-1 disks from auxiliary peg to destination peg (using source as helper).
7. Enter the query with the number of disks and peg names.
8. Display all the moves generated by the recursion.
9. Stop the program.

4. Flowchart





5. Code with Output

Code:

```
% -----  
% Towers of Hanoi  
% -----  
hanoi(0, _, _, _) :- !.  
hanoi(N, From, To, Via) :-  
    N > 0,  
    N1 is N - 1,  
    hanoi(N1, From, Via, To),  
    format("Move disk ~w from ~w to ~w~n", [N, From, To]),  
    hanoi(N1, Via, To, From).
```

Output:

```
?- hanoi(3, left, right, center).  
Move disk 1 from left to right  
Move disk 2 from left to center  
Move disk 1 from right to center  
Move disk 3 from left to right  
Move disk 1 from center to left  
Move disk 2 from center to right  
Move disk 1 from left to right  
true.
```

6. Result

The Prolog program was successfully implemented to solve the Towers of Hanoi puzzle for N disks, generating the correct sequence of moves using recursion.

7. Conclusion

Thus, the Towers of Hanoi problem was successfully solved using Prolog. The experiment demonstrates how a recursive predicate with a clear base case and recursive case can elegantly express a classical divide-and-conquer algorithm.